

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524281
Name	Kaviya.G

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I KAVIYA.G(1925254281) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:KAVIYA.G

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

1. End-to-End System Design

Design a scalable AI-based system using PySpark and RDD operations to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration

SOLUTION:

1.1 Problem Understanding & Objectives

The goal is to design a horizontally scalable AI system that ingests continuous, high-velocity social media data (posts, likes, shares, comments) and converts it into intelligent, real-time recommendations for end users. The system must handle unstructured text at internet scale, tolerate node failures without losing data, and deliver low-latency insights.

- Ingest streaming data from multiple social platforms concurrently.
- Clean, transform, and extract features from unstructured text using distributed RDD operations.
- Generate personalised recommendations using intelligent agents collaborating with the Spark engine.
- Scale elastically as data volume grows and recover gracefully from failures.

1.2 System Architecture

The architecture follows a layered Lambda-style design consisting of a Data Ingestion Layer, a Distributed Processing Layer (PySpark/RDD), an Intelligent Agent Layer, and a Serving Layer, as shown in Figure 1.1 below.

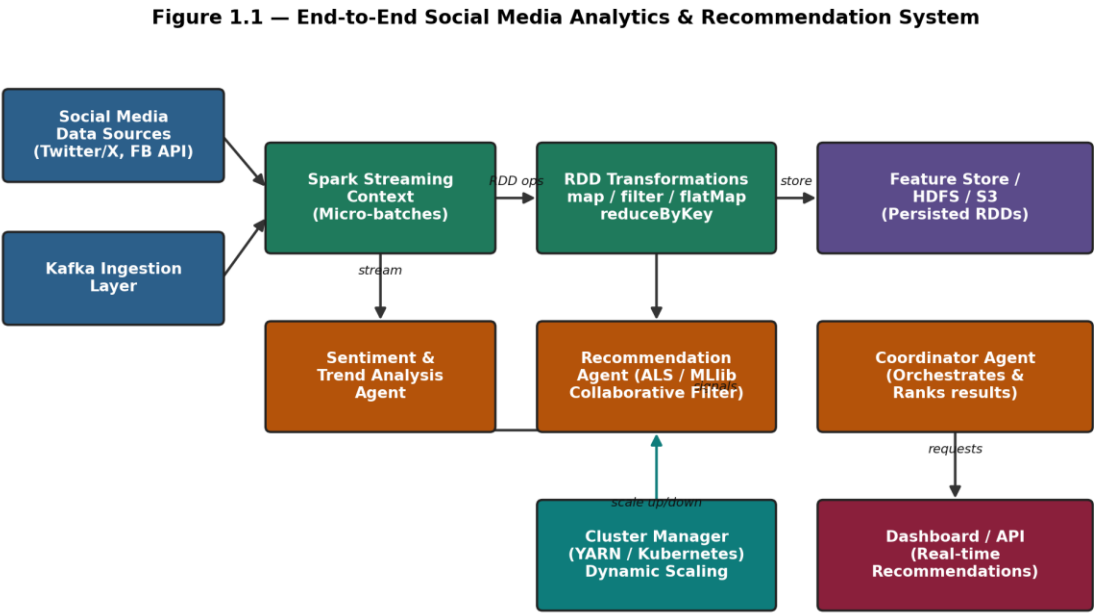


Figure 1.1: End-to-end architecture of the social media analytics and recommendation system

1.3 Component Description

Component	Technology	Responsibility
Ingestion Layer	Kafka / Flume	Buffers high-velocity streams from social APIs
Streaming Engine	Spark Structured Streaming	Converts stream into micro-batch RDDs

Component	Technology	Responsibility
Processing Layer	PySpark RDD API	Cleans, tokenizes, and aggregates text data
Sentiment Agent	Intelligent Agent	Scores sentiment/trend of each post
Recommendation Agent	MLlib ALS	Generates personalised recommendations
Coordinator Agent	Intelligent Agent	Aggregates agent outputs, resolves conflicts, ranks output
Serving Layer	REST API / Dashboard	Delivers recommendations to users in real time

1.4 PySpark / RDD Implementation

Core transformations used to process each micro-batch of social media text:

```
raw_rdd = ssc.socketTextStream(host, port)

# Cleaning & tokenizing (transformation - lazy)
tokens_rdd = raw_rdd.flatMap(lambda post: post.lower().split(' ')) \
    .filter(lambda w: w.isalpha())

# Word/topic frequency (transformation)
word_counts = tokens_rdd.map(lambda w: (w, 1)) \
    .reduceByKey(lambda a, b: a + b)

# Persist for reuse by multiple agents (action-triggering optimisation)
word_counts.persist()

# Trigger computation (action)
top_trends = word_counts.takeOrdered(10, key=lambda x: -x[1])
```

1.5 Intelligent Agent Integration

Three cooperating agents operate on top of the RDD pipeline. The Sentiment & Trend Agent consumes cleaned RDDs to classify polarity per topic; the Recommendation Agent applies collaborative filtering (ALS) on the persisted feature RDD to rank content per user; the Coordinator Agent merges both outputs, applies business rules (e.g., diversity, recency) and publishes the final ranked list to the serving layer.

1.6 Scalability & Fault Tolerance

- Data and computation are automatically partitioned across executors, enabling near-linear horizontal scaling.
- RDD lineage (DAG) allows lost partitions to be recomputed instead of restarting the whole job, providing fault tolerance without checkpoint overhead for every stage.
- Cluster manager (YARN/Kubernetes) dynamically adds/removes executors based on incoming stream load.
- In-memory caching (persist()) of frequently reused RDDs reduces recomputation cost.

1.7 Innovation

The design introduces a feedback loop where the Coordinator Agent's ranking accuracy metrics are fed back into the Recommendation Agent to continuously retrain the ALS model, creating a self-improving recommendation cycle rather than a static batch model.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

2.1 Problem Understanding & Objectives

The task requires a multi-agent AI system that uses PySpark to distribute a large-scale optimisation problem (e.g., vehicle routing / resource allocation across thousands of nodes) across a cluster, where autonomous agents operate on partitioned data and collaborate to reach a global optimum.

- Partition a large solution/search space across the cluster using RDDs.
- Assign one worker agent per partition to explore local solutions in parallel.
- Enable inter-agent collaboration and a coordinator to merge partial solutions.
- Iterate until a global convergence criterion is satisfied.

2.2 Multi-Agent System Architecture

A Coordinator–Worker pattern is used. The Coordinator Agent owns the global objective function and convergence logic; Worker Agents are deployed one per RDD partition and perform local optimisation (e.g., local search / genetic operators) in parallel across executors.

Figure 2.1 — Distributed Multi-Agent System for Large-Scale Optimization

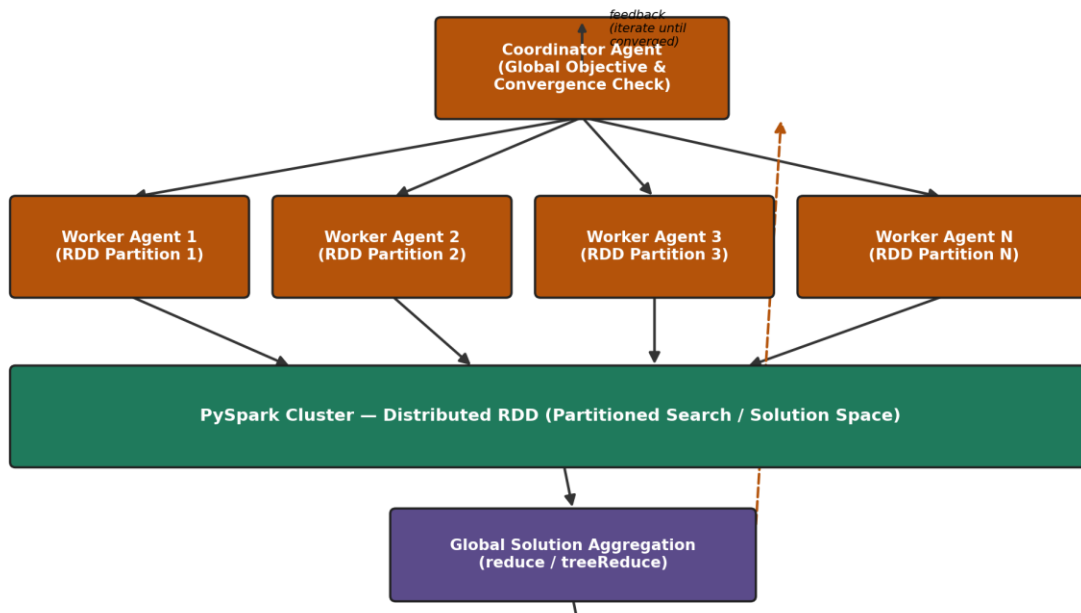


Figure 2.1: Coordinator–Worker distributed multi-agent optimisation architecture

2.3 Agent Roles

Agent	Role	Communication
Coordinator Agent	Maintains global objective, checks convergence, aggregates results	Broadcasts state to workers; receives partial solutions
Worker Agent	Explores local partition using RDD operations (local search/GA)	Reports local best solution each iteration
Aggregator	Combines worker outputs via reduce/treeReduce	Sends combined result back to coordinator

2.4 PySpark / RDD Implementation

```
# Partition the candidate solution space across the cluster
candidates_rdd = sc.parallelize(initial_population, numSlices=200)

def local_search(agent_partition):
    # Each Worker Agent explores its own partition
    best = min(agent_partition, key=fitness)
    return [best]

# Distributed local optimisation (transformation)
local_best_rdd = candidates_rdd.mapPartitions(local_search)

# Global aggregation across all worker agents (action)
global_best = local_best_rdd.treeReduce(
    lambda a, b: a if fitness(a) < fitness(b) else b)
```

2.5 Collaboration Protocol

Workers operate independently within an iteration (embarrassingly parallel), then synchronise through the Coordinator via treeReduce. The Coordinator broadcasts the updated global-best solution back to all workers (using Spark broadcast variables) so subsequent local searches are biased towards better regions of the search space — an iterative collaborate-then-converge loop.

2.6 Scalability & Fault Tolerance

- treeReduce reduces communication overhead versus a flat reduce, improving scalability for very large agent counts.
- Broadcast variables avoid re-shipping the global state to every executor on each task.
- RDD lineage recovers a failed worker's partition automatically by recomputation.

2.7 Innovation

Instead of a purely static partitioning, the design supports adaptive repartitioning — partitions that consistently underperform are merged or reseeded with the current global-best solution, accelerating convergence for highly non-uniform optimisation landscapes.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

3.1 Problem Understanding & Objectives

The requirement is a complete, fault-tolerant, distributed pipeline that takes raw big-data input through cleaning, feature engineering, and model training using RDD transformations end-to-end, so that the pipeline scales with data volume and survives executor failures.

3.2 Pipeline Architecture

Figure 3.1 — Distributed AI Pipeline using RDD Transformations

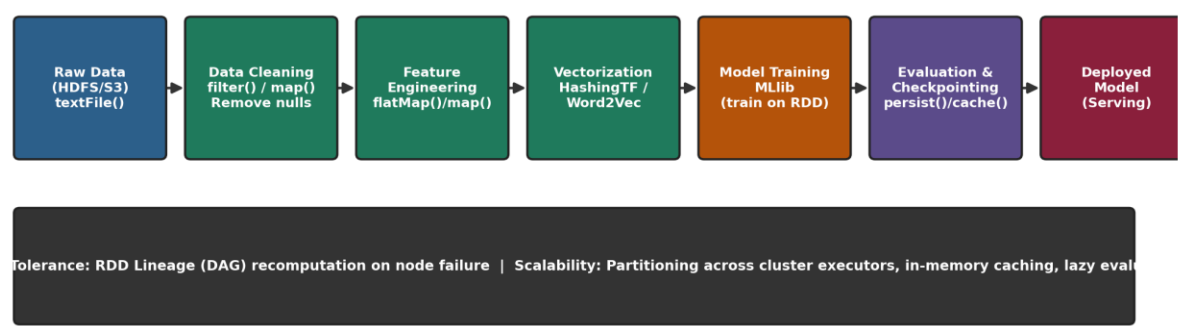


Figure 3.1: Seven-stage distributed AI pipeline built entirely from RDD transformations and actions

3.3 Stage-Wise RDD Operations

Stage	RDD Operation(s)	Purpose
Data Loading	sc.textFile()	Loads raw data as a distributed RDD across the cluster
Cleaning	filter(), map()	Removes nulls/duplicates, standardises format
Feature Engineering	flatMap(), map()	Tokenizes and derives new features per record
Vectorization	map() + HashingTF	Converts features into numeric vectors
Training	mapPartitions() + MLlib	Trains model in a distributed manner
Evaluation	reduce(), persist()	Aggregates metrics; caches intermediate RDDs

3.4 PySpark Implementation

```

raw_rdd = sc.textFile('hdfs://data/raw_dataset.csv')

# Stage 1: Cleaning (transformation)
clean_rdd = raw_rdd.filter(lambda line: line.strip() != "") \
    .map(lambda line: line.split(','))

# Stage 2: Feature engineering (transformation)
features_rdd = clean_rdd.map(lambda r: extract_features(r))

# Stage 3: Cache for repeated access by training & evaluation
features_rdd.persist(StorageLevel.MEMORY_AND_DISK)

# Stage 4: Distributed model training on partitions
model = features_rdd.mapPartitions(train_partial_model) \
    .reduce(merge_models)

# Stage 5: Evaluate (action)
accuracy = features_rdd.map(lambda x: evaluate(model, x)) \
    .mean()

```

3.5 Scalability & Fault Tolerance

- Lazy evaluation builds a DAG of transformations; nothing executes until an action is called, allowing Spark's optimizer to plan the most efficient execution.
- RDD lineage graph enables automatic recomputation of lost partitions after node failure, avoiding full pipeline restart.
- `persist(MEMORY_AND_DISK)` balances speed and reliability for datasets larger than cluster RAM.
- Checkpointing can be added for iterative ML stages to truncate very long lineage chains.

3.6 Innovation

The pipeline uses `mapPartitions()` rather than `map()` for model training so that each partition trains a lightweight local model, which are then merged (ensemble-style) — this reduces network shuffling and lets the pipeline scale to datasets far larger than a single machine's memory.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

SOLUTION:

4.1 Problem Understanding & Objectives

This system must ingest continuous streaming sensor/camera data from road junctions, process it in real time using RDD-based streaming, and let intelligent agents make split-second decisions (signal timing, rerouting, emergency priority) with minimal latency.

4.2 System Architecture

Figure 4.1 — Real-Time Decision-Making System (Smart Traffic Use Case)

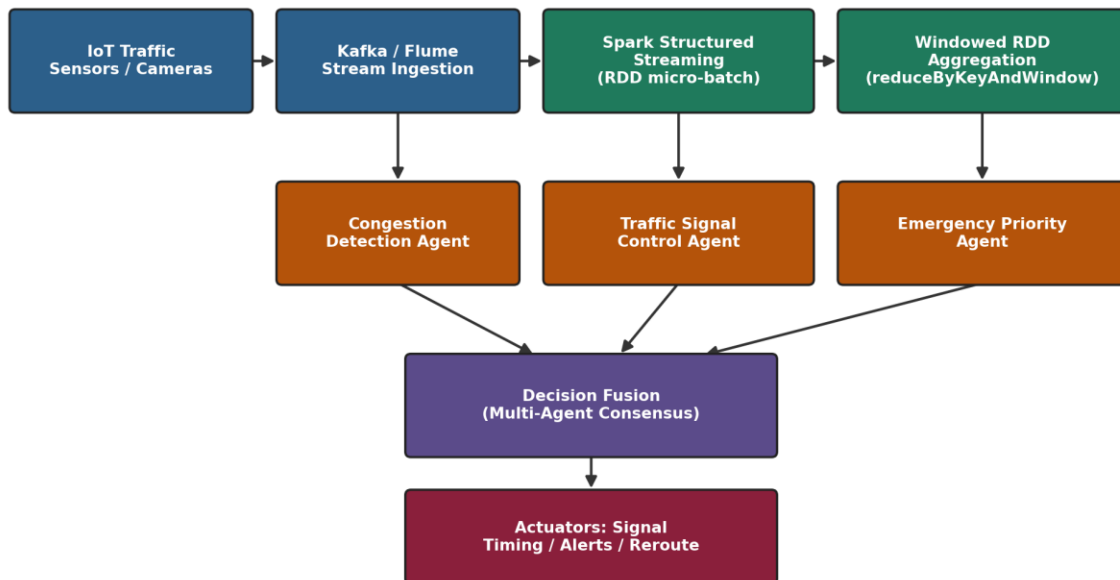


Figure 4.1: Real-time RDD streaming architecture with collaborating traffic agents

4.3 Agent Decision Table

Agent	Trigger Condition	Decision / Action
Congestion Detection Agent	Vehicle density above threshold in windowed RDD	Flags junction as congested, alerts control agent
Traffic Signal Control Agent	Congestion flag received	Recomputes green-signal duration dynamically
Emergency Priority Agent	Emergency vehicle detected in stream	Overrides signal state to grant immediate right-of-way

4.4 PySpark / RDD Streaming Implementation

```

sensor_stream = ssc.socketTextStream(host, port)

parsed_rdd = sensor_stream.map(lambda line: parse_sensor(line))

# Windowed aggregation over the last 30s, sliding every 10s
density_rdd = parsed_rdd.map(lambda r: (r.junction_id, r.vehicle_count)) \
    .reduceByKeyAndWindow(
        lambda a, b: a + b, 30, 10)

def decide(rdd):
    for junction_id, count in rdd.collect():
        if count > THRESHOLD:
            signal_agent.adjust(junction_id, count)

density_rdd.foreachRDD(decide)
  
```

4.5 Data Flow Summary

- Sensors → Kafka → Spark Streaming converts data into a continuous sequence of RDD micro-batches.
- Windowed RDD transformations aggregate vehicle counts per junction over a sliding time window.
- Agents subscribe to aggregated RDD output and independently evaluate their trigger conditions.
- A Decision Fusion step resolves conflicting agent recommendations (e.g., emergency override wins) before actuating signals.

4.6 Scalability & Fault Tolerance

Because each micro-batch is an independent RDD, a failed batch can be recomputed from its lineage without disrupting the rest of the stream. Horizontal scaling is achieved by increasing Kafka partitions and Spark executors proportionally to the number of monitored junctions, keeping end-to-end decision latency within real-time bounds (typically sub-second per micro-batch).

4.7 Innovation

An adaptive window size is used — the sliding window shrinks automatically during high-density periods (rush hour) to reduce decision latency, and widens during low traffic to save compute, giving the system context-aware responsiveness rather than a fixed batch interval.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

SOLUTION:

5.1 Problem Understanding & Objectives

The task is to design a cloud-native AI application that leverages PySpark and intelligent agents for large-scale analytics, while the cloud platform provides dynamic (elastic) resource allocation and the system continuously adapts its learning model as new data arrives.

5.2 Cloud Architecture

Figure 5.1 — Cloud-Based Distributed AI Application Architecture

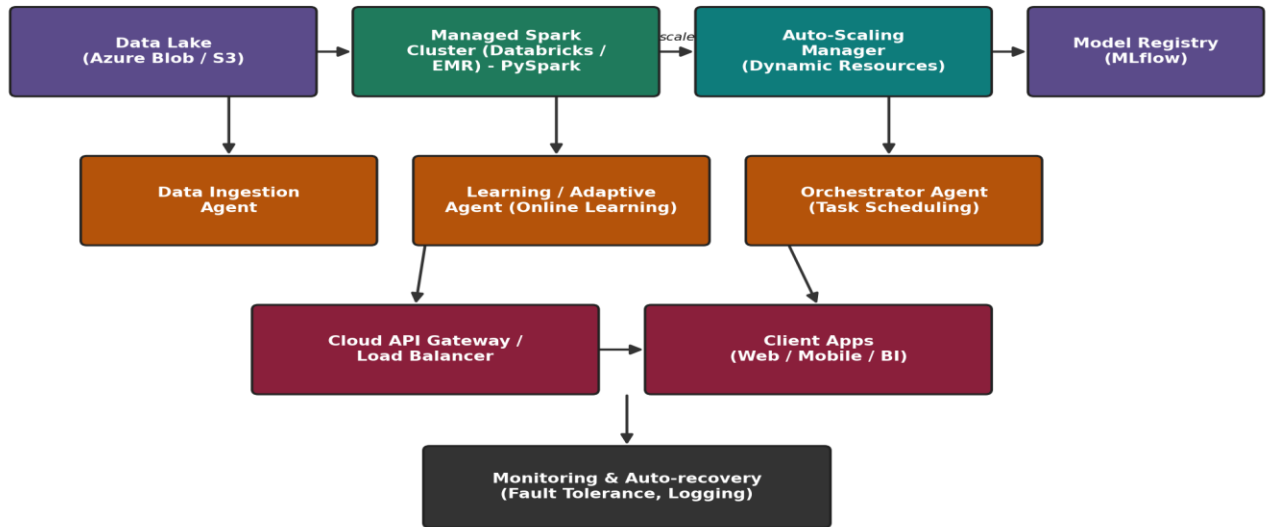


Figure 5.1: Cloud-based distributed AI application with auto-scaling and adaptive learning agents

5.3 Cloud Component Mapping

Layer	Cloud Service (example)	Function
Storage	Amazon S3 / Azure Blob Storage	Central data lake for raw and processed data
Compute	Databricks / Amazon EMR (PySpark)	Distributed RDD/DataFrame processing
Elasticity	Auto-Scaling Groups / Cluster Autoscaler	Adds/removes executors based on load
Model Ops	MLflow Model Registry	Versions and serves trained models
Access	API Gateway / Load Balancer	Exposes predictions to client applications

5.4 PySpark Implementation (Adaptive Learning Loop)

```

spark = SparkSession.builder \
    .appName('CloudAdaptiveAI') \
    .config('spark.dynamicAllocation.enabled', 'true') \
    .getOrCreate()

stream_rdd = spark.sparkContext.textFile('s3://bucket/incoming/')

features_rdd = stream_rdd.map(preprocess).cache()

# Online / incremental update of the model with new batch
def adaptive_update(rdd, model_state):
    grads = rdd.map(lambda x: compute_gradient(model_state, x))
  
```

```
avg_grad = grads.reduce(lambda a, b: a + b) / rdd.count()
return model_state - LEARNING_RATE * avg_grad

model_state = adaptive_update(features_rdd, model_state)
```

5.5 Intelligent Agent Integration

- Data Ingestion Agent: monitors the data lake and triggers new Spark jobs when fresh data arrives.
- Adaptive Learning Agent: performs incremental/online model updates so the model improves continuously instead of full batch retraining.
- Orchestrator Agent: schedules jobs across the elastic cluster and balances cost versus latency requirements.

5.6 Scalability, Elasticity & Fault Tolerance

- `spark.dynamicAllocation.enabled` lets the cluster request/release executors automatically as workload fluctuates, directly satisfying the 'dynamic resource allocation' requirement.
- Data lake storage decouples storage from compute, so compute clusters can be resized or replaced without any data loss.
- RDD lineage plus cloud-native node replacement (auto-recovery) provides two independent layers of fault tolerance.
- Monitoring and auto-recovery agent restarts failed stages and logs metrics for observability.

5.7 Innovation

The application treats model adaptation itself as an elastic workload: the Adaptive Learning Agent requests additional short-lived executors specifically during model-update windows and releases them immediately after, minimising cloud cost while still supporting near-real-time model freshness.